Université Paris Dauphine – PSL

Monte Carlo Tree Search

Classical and Advanced Monte Carlo Tree Search for Hex and Y

Students:

Léa Dodeman lea.dodeman@dauphine.eu
Nestor Antuñano Cabrera nestor.antunano-cabrera@dauphine.eu

April 8, 2026

Abstract

This repository studies Monte Carlo Tree Search across two connection games, Hex and Y, inside one shared implementation framework. The project has three active branches: a classical Hex baseline reproducing the UCT+RAVE setting of *Monte-Carlo Hex*, a learned Hex agent based on Gumbel AlphaZero, and a new classical implementation for Y. The main engineering goal is consistency: the classical branches expose the same Python and Cython APIs, use incremental connectivity tracking, and can be compared without conflating algorithmic differences with implementation asymmetries. The main research takeaway is twofold. First, the classical Hex results preserve the qualitative behavior reported in the literature even with simpler rollouts. Second, the new Y implementation now matches the Hex branch in logic and speed infrastructure. Its full-size experiments show the expected simulation-scaling curve, a strong preference for $C_{\text{uct}} = 0$, and a clear benefit from connectivity-aware rollouts, while also making clear that Y still lacks an analogue of Hex bridge knowledge with equally strong structural impact.

1 Introduction

Monte Carlo Tree Search (MCTS) is a simulation-based search procedure designed for sequential decision problems with large game trees and difficult static evaluation functions. Starting from a root state, it repeatedly alternates between selection, expansion, simulation, and backpropagation. Rather than distributing work uniformly, it allocates simulations toward the lines of play that currently appear most promising. This makes it particularly well suited to connection games, where local moves may have long-range strategic consequences and exhaustive search becomes impractical very quickly.

This project began as a study of Hex and then expanded into a broader connection-game repository. The current codebase contains three active modules. `HexClassic/` implements a classical UCT+RAVE baseline inspired by Cazenave and Saffidine's *Monte-Carlo Hex* [4]. `HexGumbel/` replaces rollout evaluation with a learned policy-value model and follows the Gumbel AlphaZero search scheme [5]. `YClassic/` extends the classical branch to the game of Y while preserving the same implementation standards as the Hex baseline.

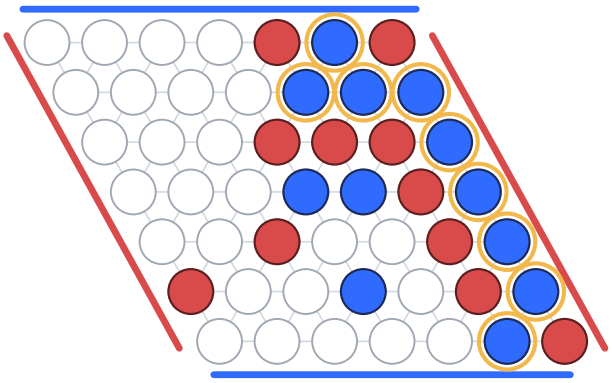
The report therefore has a double purpose. It documents the classical and learned Hex systems, but it also explains how the Y branch was upgraded from a draft into a fully usable experimental module. The central idea is that the repository should read as one coherent study, not as three unrelated projects. Shared engineering decisions are described once, and game-specific differences are explained only where the geometry, rules, or available heuristics actually diverge.

2 Literature Review and State of the Art

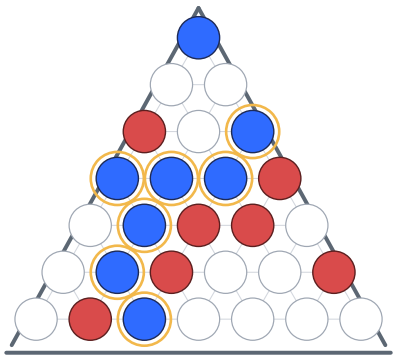
The study of Hex-playing algorithms dates back to the early history of game-playing artificial intelligence. One of the foundational discussions of computer approaches to board games appears in Shannon's classic paper on computer chess [8], which helped establish the broader idea that strategic play can be approached through search guided by approximate evaluation rather than exhaustive enumeration.

Early approaches to game playing relied on minimax search combined with alpha-beta pruning, which systematically explores the game tree while removing branches that cannot influence the final decision. While effective in games such as chess, these methods are less natural for Hex because of its large branching factor and the difficulty of designing static evaluation functions that capture global connectivity well.

Modern approaches to Hex have been largely shaped by Monte Carlo Tree Search. Cazenave and Saffidine [4] introduced a framework combining UCT with the RAVE heuristic, significantly



Hex. Example finished position on a size-7 rhombus board.



Y. Example finished position on the standard triangular board.

Figure 1: Illustrative end positions for the two connection-game geometries used in the repository. Blue and red stones correspond to the two players, and the gold outline marks one winning connected component.

improving the efficiency of search by sharing information between simulations. This paper is the direct methodological baseline for the classical branch of this repository.

Building on these ideas, Arneson, Hayward, and Henderson developed MoHex [1], a strong Hex-playing program based on MCTS that achieved top-level performance in computer Hex competitions. Subsequent work further strengthened this direction by incorporating learning components. In particular, Gao, Müller, and Hayward proposed a three-head neural architecture for MCTS in Hex [7], improving both move selection and position evaluation by enriching the information returned by the network.

More recent research has explored deeper integration of neural networks and reinforcement learning. Takada et al. introduced the DeepEzo line of work [10], which learns value and policy functions for Hex through self-play and search. Cazenave's Polygames framework [3] then showed more broadly that zero-learning style systems can be transferred across several board games within one shared framework. In parallel, AlphaGo Zero made the policy-value paradigm central to modern game AI [9], and Gumbel AlphaZero refined the root-search procedure itself [5]. These developments are the natural background for the learned Hex branch in this repository.

The Y extension occupies a slightly different position in this landscape. There is a substantial methodological literature on MCTS and a substantial applied literature on Hex, but far less standardized experimental literature on modern Y engines. For that reason, the Y branch in this repository should be read less as a reproduction of one canonical prior benchmark and more as a transfer of the connection-game search toolbox developed for Hex to a more general three-sided setting. Its rule model is grounded in standard descriptions of Y itself [2, 6], while its algorithmic foundations come primarily from the Hex and general MCTS literature above.

3 Games and Scope

3.1 Hex

Hex is played on an $N \times N$ rhombus of hexagonal cells. One player tries to connect the top and bottom borders, while the other tries to connect the left and right borders. Players alternate placing stones on empty cells, and stones are never moved or removed. The game has no draws, which is one reason it is a natural domain for Monte Carlo evaluation: every complete simulation terminates with a unique winner.

From a search perspective, Hex is difficult because its value depends on global connectivity

Module	Game	Role
`HexClassic/`	Hex	Classical UCT+RAVE baseline and reproduction study
`HexGumbel/`	Hex	Learned policy-value search with Gumbel root search
`YClassic/`	Y	Classical Y search adapted from the Hex baseline

Table 1: Active repository modules and their role in the overall study.

rather than on local material balance. A move that looks modest in isolation may become decisive once it links two larger structures elsewhere on the board. This is precisely the kind of setting in which simulation-based search became attractive.

3.2 Y

Y belongs to the same family of connection games but changes the geometry and the win condition. The version used in this repository is the standard triangular hex-grid form of Y described by Boardspace and Gambiter [2, 6]. A player wins by forming one connected component touching all three sides of the triangle. Corner cells count as belonging to both adjacent sides, and a full board has exactly one winner [2, 6]. Standard descriptions also note an alternative "official" board with three five-neighbor points, but that is not the model used here. We adopt the regular triangular grid because it provides the cleanest comparison with Hex and the most direct extension of the existing code.

3.3 Repository Modules

The repository is organized around three modules that play distinct roles in the study:

This structure makes the comparison interpretable. The two classical modules share the same general search philosophy and the same engineering backbone, while the learned Hex branch shows what changes when rollouts are replaced by a policy-value model. The Y work in the current repository is classical only, which keeps the scope of the new extension manageable and technically focused.

Figure 1 gives a visual reference for the two board geometries used throughout the report. Both examples are rendered from finished self-play positions generated with the repository's own code, and the Y figure uses the standard triangular-board presentation described in [2, 6].

4 Methods

4.1 Classical MCTS with UCT and RAVE

The classical baseline follows the UCT+RAVE structure introduced in *Monte-Carlo Hex* [4]. During selection, each action is scored by the empirical action value augmented with the usual UCT exploration bonus:

$$\mathrm{UCT}(s,a) = Q(s,a) + C_{\mathrm{uct}}\sqrt{\frac{\log N(s)}{N(s,a)}}.$$

RAVE, also known as AMAF, supplements this estimate by reusing information from later simulation moves. Informally, if a move proves good in a playout, RAVE treats it as evidence that the move might also have been good if played earlier. This is especially useful at the start of search, when ordinary visit counts are still sparse. In our implementation, the Monte Carlo estimate and the RAVE estimate are blended with the same bias-controlled interpolation used in [4].

The interesting modeling choice lies in the rollout policy. In Hex, we keep a lightweight bridge-defense heuristic rather than the stronger template stack used in the original paper. That makes the rollout policy weaker, but it also makes the behavior of the UCT+RAVE core easier to isolate. In Y, there is no direct bridge analogue, so the playout policy is based on local connectivity criteria instead. In both cases the goal is the same: keep the search logic fixed, then ask what game-specific structural knowledge matters.

4.2 Neural-Network-Guided MCTS

The learned branch replaces stochastic rollout evaluation with a neural policy-value model. At interior tree nodes, search follows the standard PUCT rule

$$Q(s,a) + c_{\text{puct}}\, P(s,a) \frac{\sqrt{N(s)}}{1 + N(s,a)},$$

where $P(s,a)$ is the prior probability predicted by the network. The network therefore affects the search in two ways: it directs exploration toward moves that look plausible and it supplies a value estimate in place of full rollouts.

At the root, the project follows the Gumbel AlphaZero policy-improvement mechanism of [5]. Candidate moves are perturbed with Gumbel noise, filtered, and then refined through sequential halving. Relative to a pure AlphaZero-style training loop [9], the present implementation adds supervised pretraining from classical Hex labels before self-play begins. This is not just a convenience. Given the available compute budget, the warm start substantially stabilizes early training and avoids the worst part of the cold-start regime.

5 Engineering Architecture

5.1 Shared Implementation Strategy

One of the most important design choices in the repository is that the two classical modules follow the same implementation pattern. Each of them exposes a pure Python reference path and a faster Cython backend with the same public API. The Python version keeps the code easy to inspect and test. The Cython version makes large experimental sweeps practical.

The central low-level issue in both Hex and Y is connectivity maintenance. The code therefore tracks components incrementally with Union-Find structures rather than recomputing graph searches after every move. In Hex, components connect to two virtual borders. In Y, the equivalent information is encoded as a three-bit side mask. This means that the underlying pattern is shared even though the rule condition changes.

The optimized backends also follow the same philosophy. Board state is stored in typed arrays, board copies use `memcpy`-style cloning, hot-loop MCTS state is flattened into arrays where possible, and experiment workers are seeded explicitly so multiprocessing remains reproducible. The point is not that Hex and Y somehow require different optimization stories. Once the shared pattern was established for Hex, the Y extension could be upgraded without becoming a one-off implementation.

5.2 Module Structure

At the repository level, the classical and learned code paths remain clearly separated. The Hex baseline module contains board logic, the classical MCTS implementation, Cython board and tree-search backends, experiment runners, and saved JSON results. The learned Hex branch adds configuration, network definitions, self-play generation, training, and evaluation utilities around a neural MCTS core. The Y branch mirrors the classical Hex layout with board logic,

Python MCTS, Cython board and MCTS backends, an experiment runner, an interactive play script, and regression tests.

This organization matters because it makes the Y work legible as an extension of the existing architecture rather than as an isolated fork. The Y code is not trying to invent a new interface; it is deliberately conforming to the one already used by the Hex baseline.

5.3 Y as an Extension of the Hex Baseline

The Y module began as a draft that still lacked the maturity of `HexClassic/`. The extension work therefore had three objectives: correctness of the game logic, parity with the Hex architecture, and practical speed. The resulting board implementation uses the regular triangular geometry, which contains

$$n = \frac{s(s+1)}{2}$$

cells for side length s. Each stone belongs to zero, one, or two sides of the triangle depending on whether it is interior, on an edge, or in a corner.

For each player, every connected component stores a three-bit mask,

$$\texttt{LEFT} = 001, \qquad \texttt{RIGHT} = 010, \qquad \texttt{BOTTOM} = 100.$$

When a move is played, it merges with neighboring friendly components and the masks are OR-combined. A win occurs as soon as one merged component reaches `111`. This makes Y win detection constant-time after each move, exactly as the Hex implementation uses incremental connectivity instead of repeated graph traversal.

The search logic also follows the Hex baseline closely. There is a single UCT+RAVE loop, standard AMAF-style updates, and full-board playouts. The one major game-specific difference is the rollout policy. Hex benefits from a simple but meaningful bridge-defense heuristic. Y has no equally direct local analogue, so its stronger rollout policy instead prefers moves that merge friendly components, increase the number of touched sides, or contest structurally important regions around opposing stones.

Correctness was checked in three complementary ways. Small full boards were enumerated to confirm that every complete coloring has exactly one winner. Random game states were compared against a naive connectivity evaluator. Finally, Python and Cython implementations were compared on identical move sequences to ensure state-by-state agreement. The result is not merely a faster draft; it is a module whose logic is now trustworthy enough to support larger experiments.

On the performance side, the gain from the fast backend is substantial. On an empty side-length-7 board with 2,000 simulations, the Python path takes 0.298 s while the Cython path takes 0.0135 s, a 22.1× speedup. More importantly for practical experimentation, the full-size matched benchmark is not dramatically more expensive than Hex. On side length 11 with the Cython backend and the same 2,000-simulation root search, Hex takes 0.0276 s and Y takes 0.0377 s. The Y extension is therefore only about 1.37× slower in the non-toy setting.

6 Experimental Results

6.1 HexClassic

The classical Hex experiments are designed as a comparative reproduction of the qualitative trends reported in *Monte-Carlo Hex* [4]. All results below use the Cython backend on 11×11 boards with 200 games per data point.

Simulations	Ours	Paper	Delta
1,000	31.5%	6.0%	+25.5
2,000	45.5%	11.5%	+34.0
4,000	41.5%	20.0%	+21.5
8,000	50.0%	33.0%	+17.0
32,000	55.5%	61.0%	-5.5
64,000	66.5%	68.5%	-2.0

Table 2: HexClassic simulation-count sweep on 11×11 boards, measured as win rate against the 16,000-simulation reference. The "Paper" column is the corresponding trend reported in *Monte-Carlo Hex*.

C_{uct}	Ours	Paper	Delta
0.0	88.0%	61.0%	+27.0
0.1	62.5%	60.0%	+2.5
0.2	52.5%	55.5%	-3.0
0.4	43.0%	42.0%	+1.0
0.5	55.5%	41.0%	+14.5
0.6	47.5%	35.5%	+12.0
0.7	50.5%	32.5%	+18.0

Table 3: HexClassic sweep over the UCT exploration constant with RAVE enabled. Each entry reports the win rate of the tested constant against the $C_{\text{uct}} = 0.3$ reference used in the paper comparison.

Simulation count. The expected monotone trend is preserved. The simplified rollout policy makes the low-simulation part of the curve stronger than the original paper's numbers in this particular comparison, but the broad behavior remains the same: larger simulation budgets improve play, and the gap narrows at high counts where tree policy matters more than rollout strength.

UCT exploration constant. The strongest setting remains $C_{\text{uct}} = 0$ when RAVE is enabled. This matches the standard interpretation from [4]: RAVE already injects substantial early exploration, so additional UCT exploration can become redundant or even harmful.

Rollout policy and RAVE bias. The Hex baseline therefore does what a baseline should do: it reproduces the main qualitative behavior of UCT+RAVE while remaining simple enough that the effect of the search algorithm is still visible.

6.2 HexGumbel

The learned Hex branch is evaluated on a smaller 7×7 board, but with a very different computational trade-off. The neural agent uses only 16 simulations per move, while the classical opponent still uses 16,000. The question is not whether the learned agent runs the same search more cheaply; it is whether a learned prior and value estimate can compensate for a drastic reduction in rollout budget.

The training curve is quite informative. The agent reaches perfect performance against random play very early, which indicates that it quickly captures basic structural regularities of Hex. More importantly, its strength against the much more heavily simulated classical baseline improves steadily and eventually reaches the 65–70% range. This is the central learned-search

Experiment	Result	Interpretation
Random vs bridge rollouts	28.5%	Bridges clearly stronger than random
$b = 0.0005$ vs 0.001	47.0%	
$b = 0.00025$ vs 0.001	49.0%	RAVE tuning is flatter than in the paper
$b = 0.000125$ vs 0.001	46.0%	

Table 4: HexClassic comparison of rollout policies and RAVE-bias settings. The random-vs-bridges row measures the weakness of purely random playouts, while the remaining rows compare candidate bias values against a $b = 0.001$ reference.

Iteration	vs Random	vs MCTS 16K	Buffer Size
5	100%	10.0%	69,220
10	100%	32.5%	132,931
20	100%	45.0%	255,492
30	100%	55.0%	121,818
40	100%	62.5%	63,061
50	100%	55.0%	61,491
60	100%	50.0%	62,208
70	100%	70.0%	188,366

Table 5: Training progression of the HexGumbel agent on 7×7 Hex. The agent uses 16 simulations per move and is evaluated against a classical Hex baseline using 16,000 simulations.

result of the project: a strong prior and value model can compensate for roughly three orders of magnitude fewer simulations.

The use of supervised pretraining is important here. Under the compute budget available for this project, a pure cold-start AlphaZero-style loop would have made the early training signal much noisier. Pretraining from expert classical labels is therefore not an incidental engineering trick but an integral part of how the learned branch becomes viable.

6.3 YClassic

The Y experiments were designed to mirror the classical Hex protocol rather than to remain in a toy calibration regime. The full suite was run with the Cython backend, side length 11, 200 games per data point, and seed 42. The experiment runner saves one JSON artifact per table together with a consolidated `all_results.jsonl` file in `YClassic/results/`.

Simulation count. This is the first reassuring sign that the Y branch is now a real experimental module. The curve is exactly what one would expect from a sound MCTS system: more simulations give stronger play, and once the simulation budget exceeds the 16,000-simulation reference the win rate moves above 50%.

UCT exploration constant. The tree-policy sweep is even more striking. As in Hex, the best setting is $C_{\text{uct}} = 0$ when RAVE is active, but in Y the preference is stronger. This suggests that, for the present Y rollout policy, AMAF already provides the dominant early exploration signal.

Rollout policy and RAVE bias. These results are important for interpreting the Y extension correctly. The branch is no longer "working" only in the weak sense that it runs. At the same non-toy board scale used for the Hex study, it shows meaningful separation between rollout

Simulations	1k	2k	4k	8k	32k	64k
Win rate vs 16K sims	2.5%	17.0%	34.0%	44.5%	55.0%	60.5%

Table 6: YClassic simulation-count sweep on side-length-11 boards, evaluated against a 16,000-simulation reference. The values come directly from the saved JSON experiment outputs under `YClassic/results/`.

C_{uct}	0.0	0.1	0.2	0.4	0.5	0.6	0.7
Win rate vs $C_{\text{uct}} = 0.3$	90.0%	76.0%	59.0%	45.5%	43.5%	45.5%	47.5%

Table 7: YClassic UCT exploration-constant sweep at 16,000 simulations. Each entry reports the win rate of the tested constant against the $C_{\text{uct}} = 0.3$ reference from the saved full run.

policies and a coherent parameter picture. At the same time, the RAVE-bias sweep is noticeably flatter than the corresponding Hex trends, which reinforces the idea that Y still lacks a structural heuristic as sharp as Hex bridge defense.

Experiment	Result	Interpretation
Random vs connectivity rollout	37.5%	Connectivity is clearly stronger
$b = 0.0005$ vs 0.001	51.5%	
$b = 0.00025$ vs 0.001	50.0%	RAVE bias is comparatively flat
$b = 0.000125$ vs 0.001	47.5%	

Table 8: YClassic comparison of rollout policy and RAVE-bias settings. The connectivity rollout is tested against the purely random rollout baseline, and the bias sweep compares candidate values against a $b = 0.001$ reference.

7 Discussion

Taken together, the three modules show three complementary points about MCTS in connection games.

First, classical search remains a strong and interpretable baseline. The `HexClassic/` results confirm that even a deliberately simplified rollout policy is enough to preserve the main UCT+RAVE trends from the literature. Second, the learned branch demonstrates that search quality is not only a matter of simulation count. Once the prior and value function are good enough, a neural agent can outperform a far more heavily simulated classical opponent. Third, extending an existing search system to a new game requires far more than changing the board geometry. The Y extension worked because the shared architecture was strong, but its heuristic landscape is clearly not the same as Hex's.

This last point is perhaps the most important engineering lesson of the project. The Y branch did not need a completely new software design. It needed the Hex architecture to be abstract enough to accommodate a different win condition while remaining concrete enough to stay fast. Once that backbone was in place, the remaining challenge was no longer engineering parity but game-specific knowledge.

8 Conclusion

This repository now contains a coherent study of Monte Carlo Tree Search across Hex and Y. The classical Hex branch reproduces the main qualitative findings of *Monte-Carlo Hex*. The learned Hex branch shows that Gumbel AlphaZero can outperform a classical baseline despite

using far fewer simulations per move. Most importantly for the present extension, the Y branch is no longer a draft. It now has correct rule handling, a Python/Cython structure matching the Hex baseline, regression coverage, saved experiment outputs, and a fast backend suitable for full-size sweeps.

The full Y experiments also change the interpretation of the module. The branch does not merely pass sanity checks; it already shows the expected simulation scaling, a clear preference for $C_{\text{uct}} = 0$ under RAVE, and a substantial gap between random and connectivity-aware playouts. The next step is therefore not more implementation cleanup but better Y-specific knowledge. The infrastructure is in place. What remains open is which structural heuristics or priors will do for Y what bridges do for Hex.

References

[1] B. Arneson, R. B. Hayward, and P. Henderson. Monte carlo tree search in hex. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(4):251–258, 2010.

[2] Boardspace.net. Game of y. https://www.boardspace.net/english/about_y.html. Accessed 2026-04-08.

[3] T. Cazenave, Y.-C. Chen, G.-W. Chen, S.-Y. Chen, X.-D. Chiu, J. Dehos, M. Elsa, Q. Gong, H. Hu, V. Khalidov, C.-L. Li, H.-I. Lin, Y.-J. Lin, X. Martinet, V. Mella, J. Rapin, B. Roziere, G. Synnaeve, F. Teytaud, O. Teytaud, S.-C. Ye, Y.-J. Ye, S.-J. Yen, and S. Zagoruyko. Polygames: Improved zero learning. *ICGA Journal*, 42(4):244–256, 2020.

[4] T. Cazenave and A. Saffidine. Monte-carlo hex. In H. J. Herik, H. Iida, and A. Plaat, editors, *Computers and Games: 7th International Conference, CG 2010, Kanazawa, Japan, September 24–26, 2010, Revised Selected Papers*, volume 6515 of *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 2011.

[5] I. Danihelka, A. Guez, J. Schrittwieser, and D. Silver. Policy improvement by planning with gumbel. In *International Conference on Learning Representations*, 2022.

[6] Gambiter. Y - game. https://gambiter.com/connection/Y_game.html. Accessed 2026-04-08.

[7] C. Gao, M. Müller, and R. Hayward. Three-head neural network architecture for monte carlo tree search. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, pages 3762–3768, 2018.

[8] C. E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.

[9] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.

[10] K. Takada, H. Iizuka, and M. Yamamoto. Reinforcement learning to create value and policy functions using minimax tree search in hex. *IEEE Transactions on Games*, 12(1):63–73, 2020.